

РОЗДІЛ 5: ЗАДАЧІ БЛИЗЬКОСТІ

ПОСТАНОВКА ЗАДАЧ БЛИЗЬКОСТІ

Задача Б.1 (НАЙБЛИЖЧА ПАРА). В евклідовому E^d просторі задана множина S із N геометричних об'єктів (на площині нехай це множина точок). Необхідно знайти найближчу пару (два) об'єктів (точок), тобто два об'єкти (точки), відстань між якими найменша (може виявитись, що таких пар може бути декілька, тоді достатньо знайти хоча б одну із них).

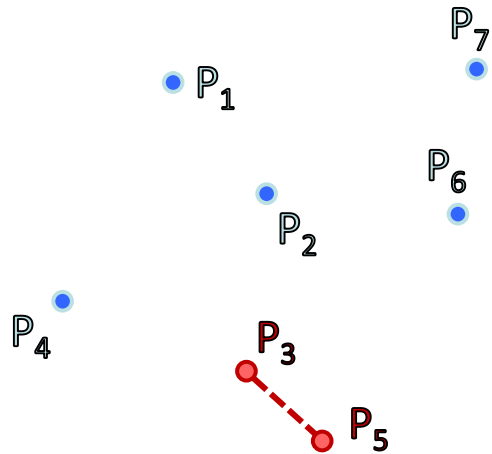
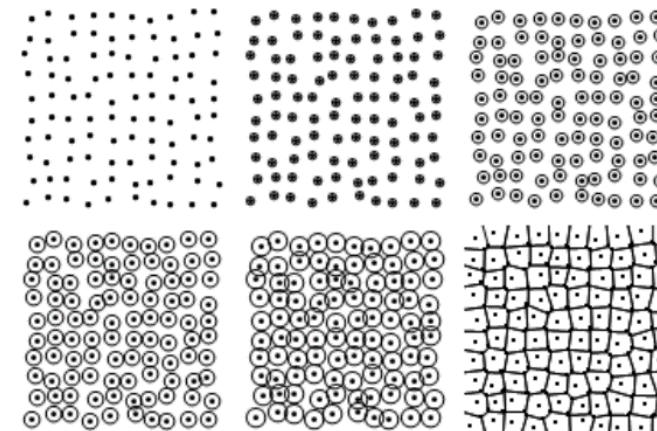


Рис. 5.1. а) найближча пара - (P_5, P_3) ;



ПОСТАНОВКА ЗАДАЧ БЛИЗЬКОСТІ

Задача Б.2 (УСІ НАЙБЛИЖЧІ СУСІДИ). В евклідовому E_d просторі задана множина S із N геометричних об'єктів (на площині нехай це множина точок). Необхідно знайти найближчого сусіда для кожного об'єкта (точки) множини S .

Означення 6.1. "Найближчий сусід" – це відношення на множині точок S , яке визначається таким чином: точка b є найближчим сусідом точки a (позначається $a \rightarrow b$), якщо $\text{dist}(a,b) = \min_{c \in S - a} \text{dist}(a,c)$,

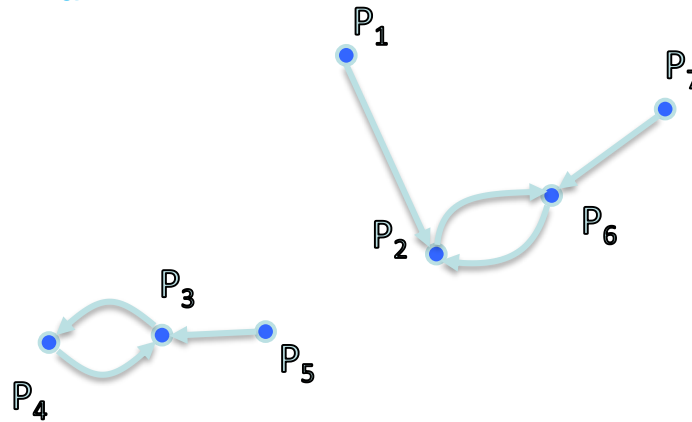
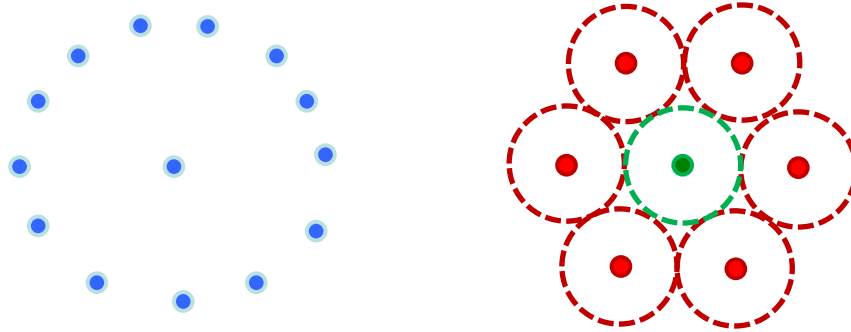


Рис. 5.1.б) Найближчі сусіди - (P_1, P_2) , (P_2, P_6) , (P_6, P_2) , (P_7, P_6) , (P_3, P_4) , (P_4, P_3) , (P_5, P_3) ; взаємні пари - (P_2, P_6) , (P_6, P_2) ; (P_3, P_4) , (P_4, P_3) .

Означення 6.2. Пара точок, яка задовольняє умові симетричності відношення називається взаємною парою.

Властивості відношення “найближчий сусід”

1. Відношення в загальному випадку не є симетричним.
2. Окрема точка на множині S із N точок може мати $N - 1$ найближчого сусіда.



3. Окрема точка на множині S із N точок може бути найближчим сусідом не більше ніж для 6 точок на площині та не більше 12 у тривимірному просторі. Це число не відоме для просторів розмірності $d > 12$.
4. Якщо процедура вибору точок множини S здійснюється по нормальному закону, то математичне сподівання долі взаємних пар точок буде рівне
$$E(S) = 6\pi / (8\pi + 3(3)^{1/2}) \sim 0.6215 \text{ (число Піелу)}.$$
5. Застосування (математична екологія). У математичній екології ця величина використовується для виявлення тенденції на виживання живих організмів деякого біологічного виду у заданих екологічних умовах.

- **Задача Б.3 (ЕВКЛІДОВЕ МІНІМАЛЬНЕ КІСТЯКОВЕ ДЕРЕВО).** На площині задано **N** точок. Побудувати дерево, вершинами якого є усі задані точки і сумарна довжина усіх ребер якого мінімальна.

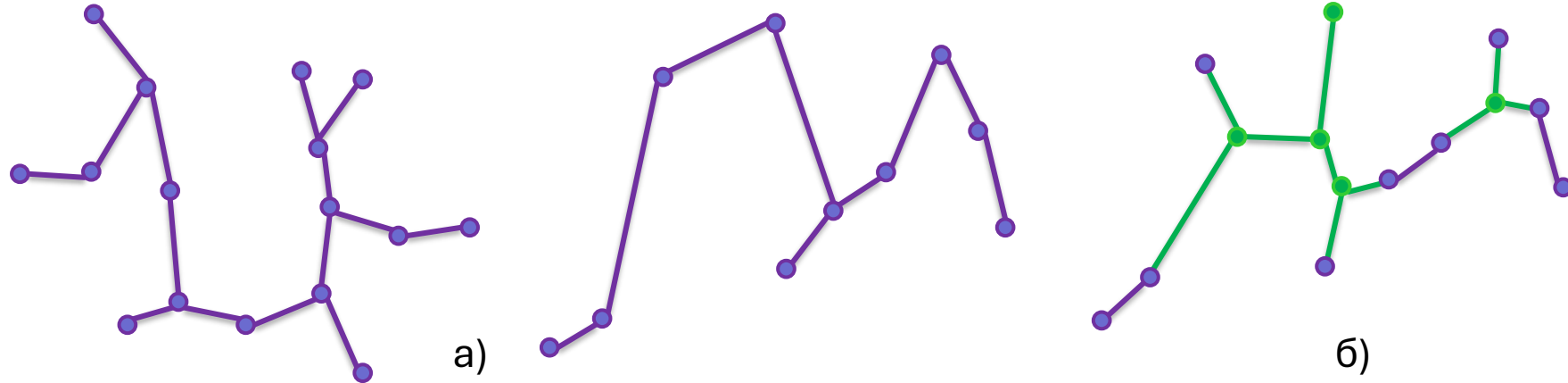


Рис.5.2. а) Мінімальне кістякове дерево множини точок на площині; б) Дерево Штейнера.

- **Задача про МКД.** Задано зважений граф з **N** вершинами і **E** ребрами, необхідно знайти найкоротше піддерево графа **G**, яке включає усі його вершини.
- **Означення 6.3.** Дерево, яке має найкоротшу довжину і задовольняє умові, що до вихідної множини не заборонено додавати нові точки, називається деревом Штейнера (Рис. 5.2 б.).

- **Задача Б.4 (ТРИАНГУЛЯЦІЯ).** На площині задано N точок. З'єднати їх відрізками таким чином, щоб кожна область всередині опуклої оболонки цієї множини точок являлась трикутником (рис. 5.3).

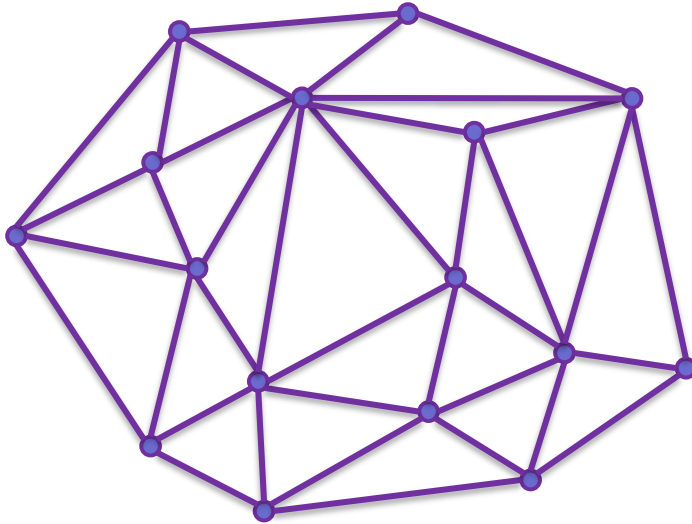


Рис. 5.3

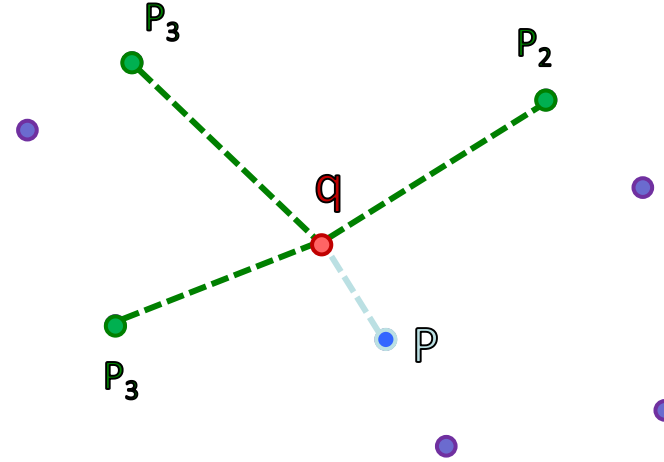


Рис. 5.4

- **Задача Б.5 (ПОШУК НАЙБЛИЖЧОГО СУСІДА).** На площині задано N точок. Як швидко можна знайти найближчого сусіда для деякої нової точки q , при умові що допускається попередня обробка?
- У просторі розмірності d цю задачу можна розв'язати за час $O(dN)$.
- **Задача Б.6 (k - НАЙБЛИЖЧИХ СУСІДІВ).** На площині задано N точок. Як швидко можна знайти k – точок, найближчих до деякої нової точки q , при умові що допускається попередня обробка?

НИЖНІ ОЦІНКИ СКЛАДНОСТІ

Означення 6.4. Задачі, з визначеними оцінками складності, які можна звести до розглядуваних називаються задачами – прототипами. Вони є базовими для певних класів.

Задача.(УНІКАЛЬНІСТЬ ЕЛЕМЕНТІВ). Задані N дійсних чисел. Визначити, чи існують серед них хоча б два рівних.

Теорема 6.1. В рамках моделі алгебраїчних дерев обчислень будь-який алгоритм, який визначає, чи є елементи множини із N дійсних чисел різними, вимагає $\Omega(N \log N)$ перевірок.

Теорема 6.2. Задача про унікальність елементів $O(N)$ зводна до задачі про найближчу пару, а отже є задачею прототипом. **УНІКАЛЬНІСТЬ ЕЛЕМЕНТІВ $\propto_N$ НАЙБЛИЖЧА ПАРА.**

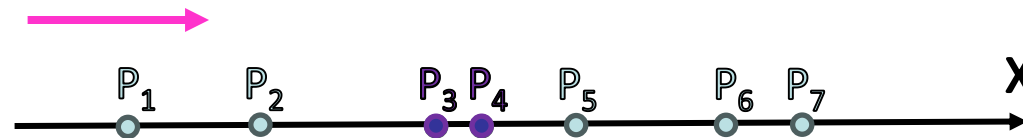


Рис. 5.5.

Доведення.

1. Нехай $A = \{x_1, \dots, x_N\}$ множина вхідних даних задачі **УЕ**.
2. $\forall x_i \in A : P_i(x_i, 0) - x_i \rightarrow (x_i, 0)$. $(\{x_1, \dots, x_N\} \propto_N \{(x_1, 0), \dots, (x_N, 0)\})$, рис. 5.5.
3. Розв'язуємо задачу про **НП**. Рухаючись по списку точок $B = \{P_1, P_2, \dots, P_N\}$ визначаємо відстань δ між сусідніми точками P_i, P_{i+1} .
4. Якщо $\delta = 0 \Rightarrow$ "НІ", $\delta \neq 0 \Rightarrow$ "ТАК".

НИЖНІ ОЦІНКИ СКЛАДНОСТІ

Теорема 6.3. Нижня оцінка складності задачі про найближчу пару рівна $\Omega(N \log N)$.

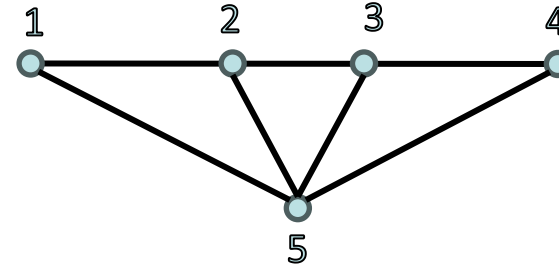
Теорема 6.4. Задача **НАЙБЛИЖЧА ПАРА** $\propto_{O(N)}$ **УСІ НАЙБЛИЖЧІ СУСІДИ** і задача **НАЙБЛИЖЧА ПАРА** $\propto_{O(N)}$ **ЕМКД**, а отже задача **НАЙБЛИЖЧА ПАРА** є їх прототипом.

Теорема 6.5. Задача **СОРТУВАННЯ** $\propto_{O(N)}$ **ЕМКД** і задача **СОРТУВАННЯ** $\propto_{O(N)}$ **ТРИАНГУЛЯЦІЯ**, а отже задача **СОРТУВАННЯ** є їх прототипом.



ЕМОД: (1,2), (2,3), (3,4), (4,5) $\Rightarrow$ (1,2,3,4,5)

Рис. 5.6



ТРИАНГУЛЯЦІЯ: (1,2,5), (2,3,5), (3,4,5) $\Rightarrow$ (1,2,3)

Рис. 5.7

Доведення. ▼ (Методом звідності)

Нехай задані N додатних дійсних чисел $S_1 = x_1, x_2, \dots, x_N$.

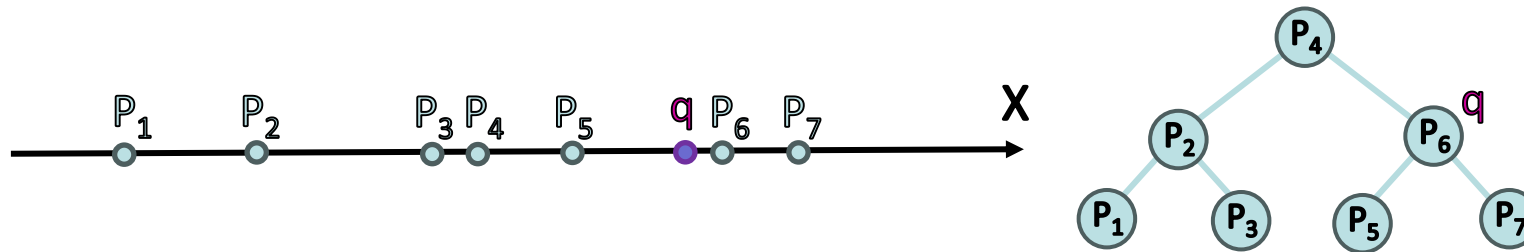
1. Позначимо **СОРТ**(S_1) задачу сортування множини S_1 , а **ЕМКД**(S_2) – задачу побудови ЕМКД.
2. Перевіримо звідність **СОРТ**(S_1) $\propto_N$ **ЕМКД**(S_2).
3. Перевіримо перетворення вхідних даних S_1 задачі **СОРТ**(S_1) у вхідні дані S_2 задачі **ЕМКД**(S_2). Для цього поставимо у відповідність $\forall x_i \in S_1$ точку $P_i(x_i, 0) \in S_2$ на числовій осі OX ($x_i \rightarrow (x_i, 0)$): $\{x_1, \dots, x_N\} \propto_N \{(x_1, 0), \dots, (x_N, 0)\}$.
4. Розв'язуємо задачу побудови **ЕМКД**(S_2).
5. Розв'язком задачі **ЕМКД**(S_2) буде список $N-1$ ребер $e_1, e_2, \dots, e_{N-1}$, де $e_j = \{(x_j, 0), (x_{j+1}, 0)\}$, $\forall j = 1, 2, \dots, N-1$. Не важко перетворити цей список в упорядкований список чисел x_j , витративши на це часу $O(N)$. Що й доводить звідність **СОРТ**(S_1) $\propto_N$ **ЕМКД**(S_2).

Аналогічно доводиться звідність **СОРТУВАННЯ** $\propto_{O(N)}$ **ТРИАНГУЛЯЦІЯ**.

Теорема 6.6. В рамках моделі дерев обчислень будь-який алгоритм, який розв'язує одну із задач - **НАЙБЛИЖЧА ПАРА**, **ЕМКД**, **ТРИАНГУЛЯЦІЯ**, **УСІ НАЙБЛИЖЧІ СУСІДИ**- вимагає $\Omega(N \log N)$ операцій.

Теорема 6.7. Задача **ДВІЙКОВИЙ ПОШУК** $\propto_{O(1)}$ **НАЙБЛИЖЧИЙ СУСІД**, а отже є її прототипом.

Доведення. Нехай задано N дійсних чисел $x_1, \dots, x_N$. В результаті двійкового пошуку визначається число x_i , найближче до числа q , яке задається в запиті на пошук. Але цю саму задачу можна подати і в геометричному формулюванні, поставивши у відповідність кожному числу x_i точку на осі OX з координатами $(x_i, 0)$. І таким чином, пошук найближчого сусіда приведе до тієї ж відповіді, що і двійковий пошук.



Теорема 6.8. Задача **НАЙБЛИЖЧИЙ СУСІД** $\propto_{O(1)}$ **k-НАЙБЛИЖЧИХ СУСІДІВ**, а отже є її прототипом.

Доведення. Що стосується задачі **k - НАЙБЛИЖЧИХ СУСІДІВ**, то, поклавши $k = 1$, відразу одержимо зведення **ПОШУК НАЙБЛИЖЧОГО СУСІДА** $\propto$ **k- НАЙБЛИЖЧИХ СУСІДІВ**.

Теорема 6.9. Нижня оцінка складності задач **ПОШУК НАЙБЛИЖЧОГО СУСІДА** та **k- НАЙБЛИЖЧИХ СУСІДІВ** для нової точки q на множині S із N точок у просторі E^d рівна $\Omega(\log N)$.

ЛЕКЦІЯ 9: МЕТОДИ РОЗВ'ЯЗУВАННЯ ЗАДАЧ НА БЛИЗЬКІСТЬ

Метод «розділяй та пануй» пошуку Найближчої Пари

Вибір стратегії вирішення проблеми

Для побудови алгоритмів з оцінкою складності $O(N \log N)$ є два шляхи: сортування і використання методу "розподіляй та пануй".

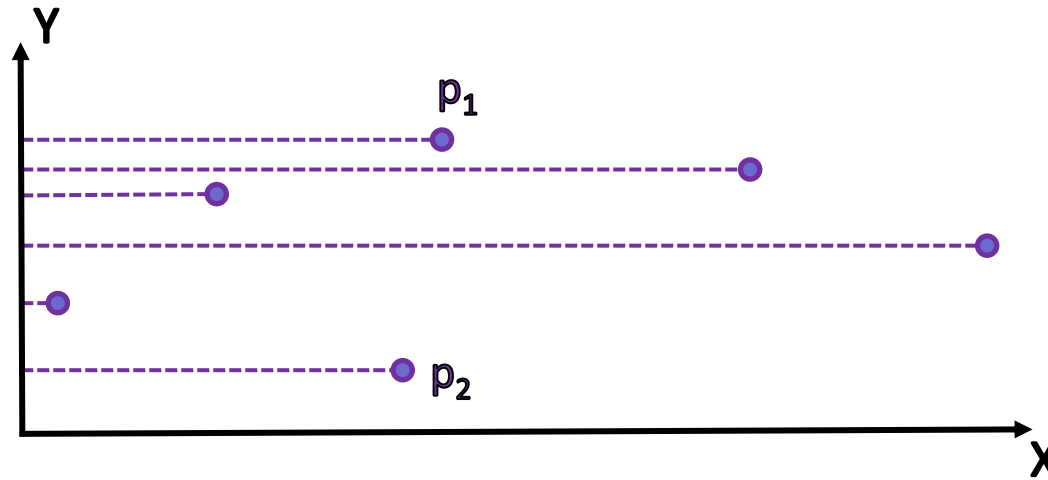


Рис. 5.8. Точки p_1 і p_2 , які утворюють найближчу пару мають найбільшу відстань по у- координаті.

Алгоритм “розділяй та пануй” (Одновимірний випадок).

Стратегія «Розділяй та пануй».

Застосування стратегії «розділяй та пануй» для розв’язання проблеми має сенс, якщо крок злиття виконується за лінійний час.

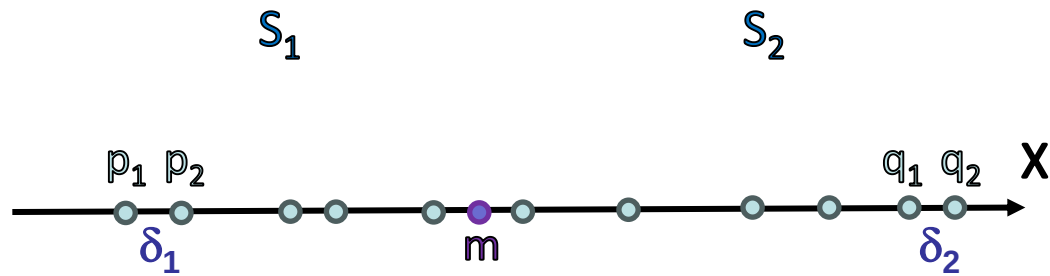


Рис.5.9.

Нехай на осі **OX** задана множина **S** із **N** точок, рис 5.9. Знайдемо найближчу пару.

1. Упорядкуємо задану множину точок по **x** - координаті.
2. Нехай точка **m** розбиває множину на дві підмножини **S1** та **S2**.
3. Знайдемо рекурсивно найближчу пару для множин **S1({p1, p2})** та **S2({q1, q2})**, які утворюють відстані **delta1**, **delta2** відповідно.

4. Позначимо через δ найменшу серед знайдених δ_1 і δ_2 відстань:
 $\delta = \min(\delta_1, \delta_2) = \min(|p_2 - p_1|, |q_2 - q_1|)$.
5. Відкладемо ліворуч і праворуч відносно точки m відрізки довжиною δ .
6. Для точок із множин $S_1(p_3)$ та $S_2(q_3)$, які потрапили у δ -окіл точки m знайти найближчу пару:
 $\text{dist}(\max(S_1), \min(S_2)) = |p_3 - q_3| = \delta_3$
7. Порівнявши відстані (δ_3), знайдених на кроці 6 пар, з відстанню δ остаточно знайдемо найближчу пару:
 $\delta^* = \min(\delta = \min(\delta_1, \delta_2), \text{dist}(\max(S_1), \min(S_2))) = \min(\delta, \delta_3)$

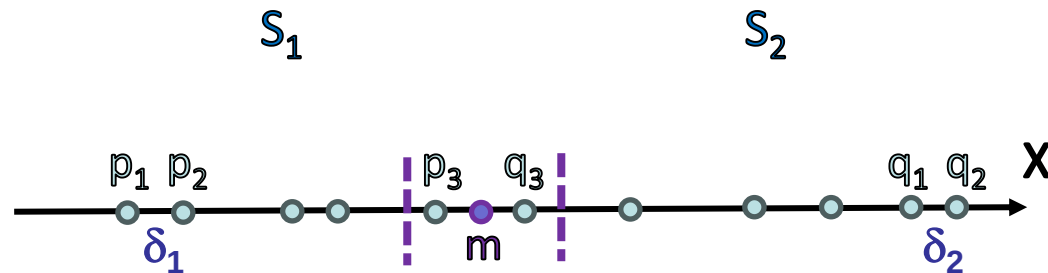
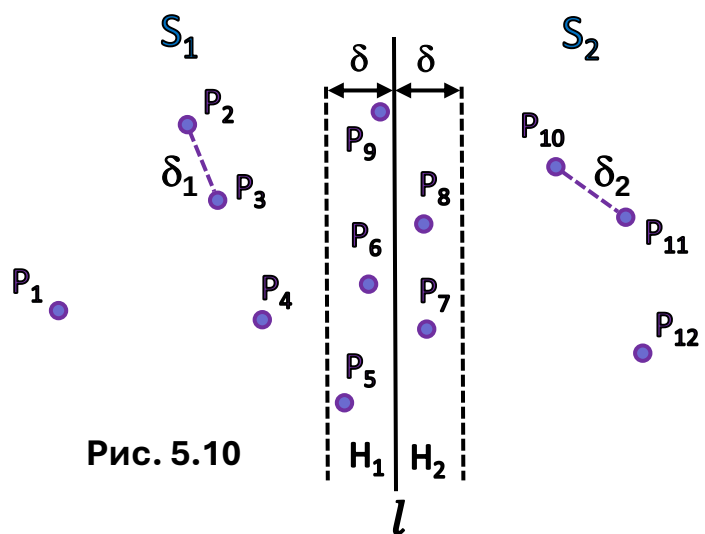


Рис. 5.9.

Таким чином отримаємо алгоритм знаходження найближчої пари для одновимірного випадку зі складністю $\theta(N \log N)$.

Алгоритм “розділяй та пануй” (двовимірний випадок)

1. Спочатку побудуємо відсортований список точок по x координаті $S_x = \{P_{1x}, P_{2x}, \dots, P_{Nx}\}$, а потім відсортований список точок по y координаті $S_y = \{P_{1y}, P_{2y}, \dots, P_{Ny}\}$.
2. На основі списків S_x, S_y формуємо масив точок $U = \{P_{ij} \mid i, j = 1, N\}$, де i - індекс, який указує номер точки у впорядкованому списку S_x , а j - індекс, який указує номер точки у впорядкованому списку S_y .
3. **Розбиття множини точок (рекурсивний спуск).** Цей етап алгоритму полягає у розбитті на кожному кроці рекурсії заданої множини у вигляді списку S на рівнопотужні підмножини S_1, S_2 вертикаллю l (медіаною по x координаті).
4. Розв’язавши рекурсивно задачу для S_1 і S_2 , одержимо найближчі пари **НП1** і **НП2** та δ_1, δ_2 - мінімальні відстані, які вони утворюють для множин S_1 і S_2 , відповідно (рис.5.10).
5. Залишається перевірити найближчі пари, які можуть бути на границях множин S_1, S_2 . Для цього покладемо $\delta = \min(\delta_1, \delta_2)$. Відкладемо ліворуч і праворуч від медіани l смуги H_1 і H_2 шириною δ .



Алгоритм “розоділяй та пануй” (двовимірний випадок)

6. **Злиття (рекурсивний підйом).** Цей етап полягає у рекурсивному визначенні на кожному кроці найближчої пари серед точок множин S_1 , S_2 які потрапили у смуги, відкладені ліворуч і праворуч від вертикалі l (H_1 і H_2 , відповідно) шириною δ .

6.1. Нехай H_1^* і H_2^* упорядковані по зростанню y - координати списки проекцій точок, які потрапили у смуги H_1 і H_2 .

6.2. Рухаючись по списку H_1^* , для кожної його точки із списку H_2 роглядаються точки, які потрапляють у прямокутник $\delta \times 2\delta$ відносно проекції цієї точки у смугі H_2 . Максимальна кількість таких точок буде не більша ніж 6. Тому на кроці злиття розв'язків підзадач необхідно виконати не більше $6 \times N / 2 = 3N$ порівнянь відстаней.

7. Відстані знайдених найближчих пар на кроці 6 порівнюються із відстанню δ , знайдену на кроці 5:
 $\delta^* = \min(\delta = \min(\delta_1, \delta_2), \text{dist}(\max(S_1), \min(S_2))) = \min(\delta, \delta_k), \forall k$

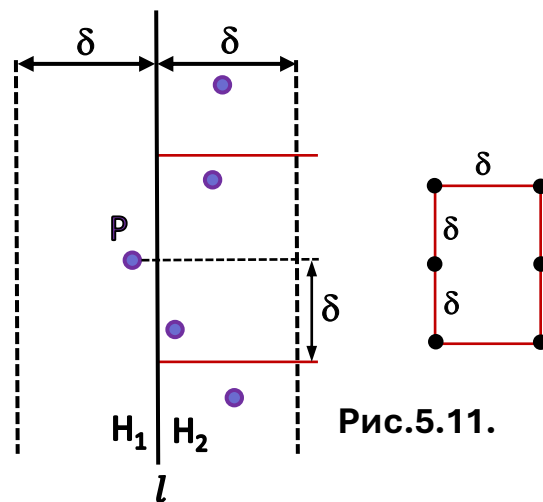


Рис.5.11.

Теорема 6.10. Найкоротша відстань, яка визначається N точками на площині, може бути знайдена методом «розділяй та пануй» за час $\theta(N \log N)$ і є оптимальна.

- **function** НПАРА1(S)
- **begin if** ($| S | = 2$) **then** $\delta := | X[2] - X[1] |$
- **else if** ($| S | = 1$) **then** $\delta := \infty$
- **else begin** $m := \text{Медіана}(S);$
- Побудувати ($S1, S2$) ($*S1 = \{p: p \leq m\}, S2 = \{p: p > m\}*$);
- $\delta1 := \text{НПАРА}(S1);$
- $\delta2 := \text{НПАРА}(S2);$
- $p := \max(S1);$
- $q := \min(S2);$
- $\delta := \min(\delta1, \delta2, q - p)$
- **end;**
- **return** δ
- **end.**

Метод «розділяй та пануй» розв'язання задачі пошуку найближчої пари.

Приклад.

1. Вводимо точки, рис.1: $S = \{P_1, P_2, P_3, P_4, P_5, P_6, P_7, P_8, P_9, P_{10}, P_{11}, P_{12}\}$

2. Формуємо відсортовані списки точок по x та y координатах:

$$U_x = \{P_5, P_6, P_9, P_1, P_2, P_{12}, P_{10}, P_7, P_{11}, P_4, P_3, P_8\},$$

$$U_y = \{P_6, P_{11}, P_{12}, P_9, P_8, P_4, P_{10}, P_5, P_1, P_7, P_3, P_2\}.$$

На основі списків U_x , U_y формуємо масив точок $U = \{P_{ij}, i, j = 1, N\}$, де i - індекс, який указує номер точки у впорядкованому списку U_x , а j - індекс, який указує номер точки у впорядкованому списку U_y .

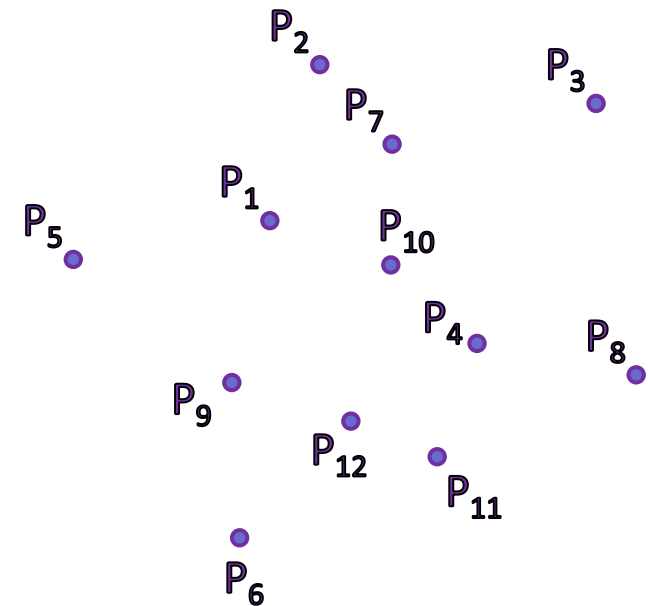


Рис.1

Метод «розділяй та пануй» розв'язання задачі пошуку найближчої пари

3. Розбиття множини точок (рекурсивний спуск).

3.1. Рекурсивне розбиття на кожному кроці заданої множини S у вигляді списку

$$U_x = \{P_5, P_6, P_9, P_1, P_2, P_{12}, P_{10}, P_7, P_{11}, P_4, P_3, P_8\},$$

на рівнопотужні підмножини U_1, U_2 вертикаллю l . Для цього знаходиться медіана списку (точка P_{12}). Вертикаль l проведемо посередині між медіаною і сусідньою точкою P_{10} , рис. 2. Розбиття подаємо у вигляді дерева, рис.3.

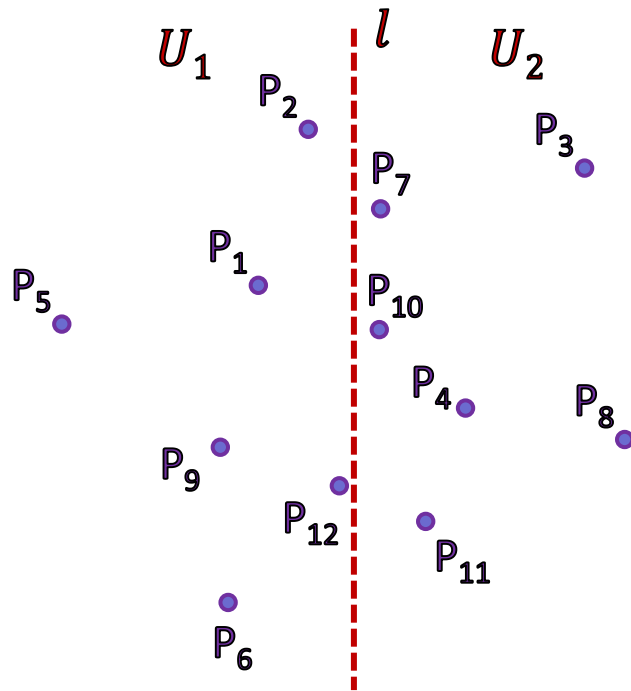
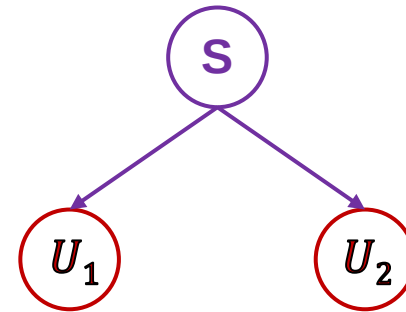


Рис.2.

$$\{P_5, P_6, P_9, P_1, P_2, P_{12}, P_{10}, P_7, P_{11}, P_4, P_5, P_8\}$$



$$\{P_5, P_6, P_9, P_1, P_2, P_{12}\} \quad \{P_{10}, P_7, P_{11}, P_4, P_5, P_8\}$$

Рис.3.

Метод «розділяй та пануй» розв'язання задачі пошуку найближчої пари

3. Розбиття множини точок (рекурсивний спуск).

3.2. Далі, аналогічно, розіб'ємо множини U_1 , U_2 вертикалями l_1 і l_2 на рівнопотужні підмножини $U_1 = U_{11} \cup U_{12}$, $U_2 = U_{21} \cup U_{22}$ відповідно, рис.4. Маємо дерево, рис.5.

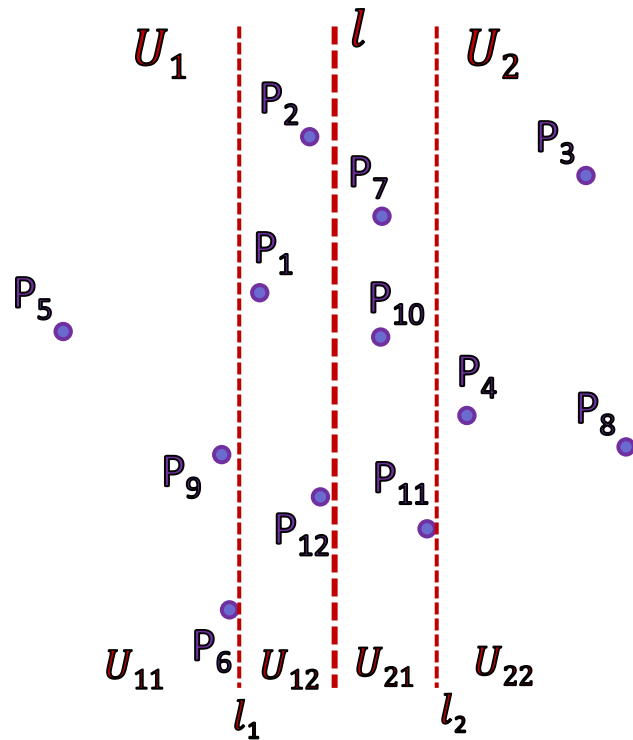


Рис.4.

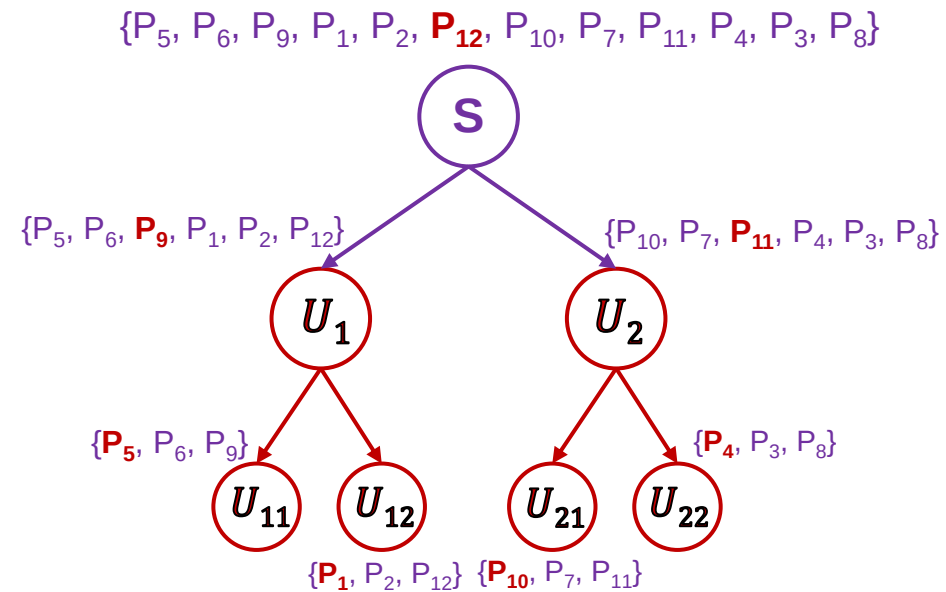


Рис.5.

Метод «розділяй та пануй» розв'язання задачі пошуку найближчої пари

4. Злиття результатів (рекурсивний підйом).

4.1 Будемо вважати, що ми рекурсивно знайшли найближчі пари і відстані між ними для кожного (окрім останнього) вузла (і відповідної множини точок) дерева алгоритму: $U_{11}(\delta_{9,6})$, $U_{12}(\delta_{1,2})$, $U_{21}(\delta_{10,7})$, $U_{22}(\delta_{4,8})$; $U_1(\delta_{9,12})$, $U_2(\delta_{10,7})$, рис. 6.

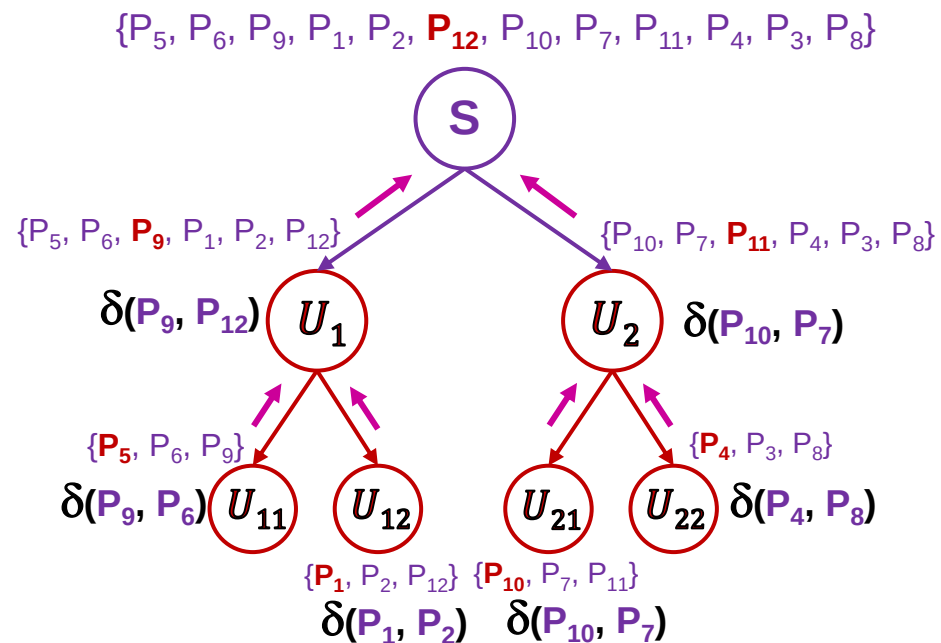


Рис.6.

Метод «розділяй та пануй» розв'язання задачі пошуку найближчої пари

4.2. Покажемо детально останній крок злиття для вузла S , вважаючи, що ми уже рекурсивно знайшли найближчу пару для множини точок списків $U_1(\text{НП}(P_9, P_{12}), \delta_{9,12})$, $U_2(\text{НП}(P_{10}, P_7), \delta_{10,7})$, відповідно. Залишилось перевірити точки на границях множин точок списків U_1 і U_2 .

4.3. Визначимо зону пошуку найближчих пар на границях множин U_1 і U_2 . Для цього візьмемо $\delta = \min(\delta_{9,12}, \delta_{10,7})$ і відкладемо ліворуч і праворуч від вертикалі l смуги (H_1 і H_2 , відповідно) шириною δ .

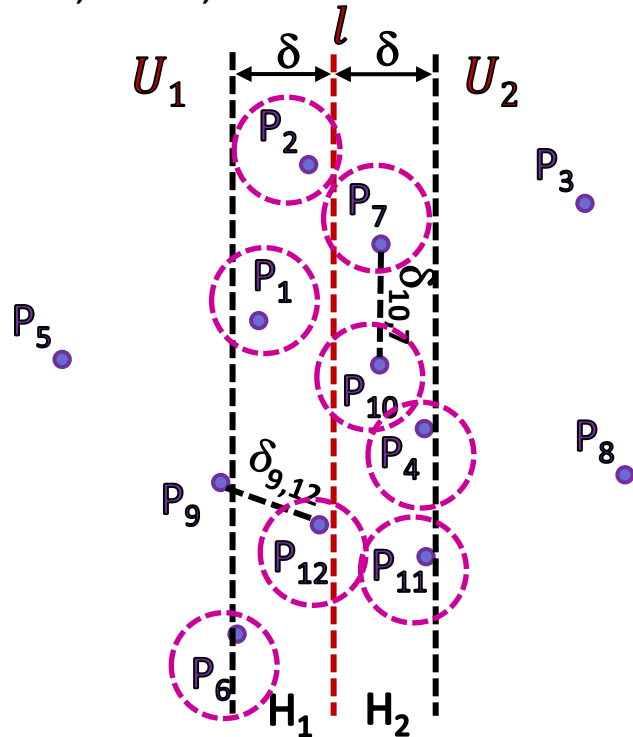


Рис.7.

4.4. Нехай H_1^* і H_2^* упорядковані по зростанню y - координати списки точок, які потрапили у смуги H_1 і H_2
 $H_1^* = \{P_6, P_{12}, P_1, P_2\}$, $H_2^* = \{P_{11}, P_4, P_{10}, P_7\}$.

4.5. Рухаючись по списку H_1^* (у порядку зростання y координати) для кожної його точки досліджуємо точки із списку H_2^* , які потрапили у прямокутник $\delta \times 2\delta$ (утворений відкладеною відстанню δ вгору і вниз від y - координати розглядуваної точки та шириною смуги H_2). Максимальна кількість таких точок буде не більша ніж 6.

Метод «розділяй та пануй» розв'язання задачі пошуку найближчої пари

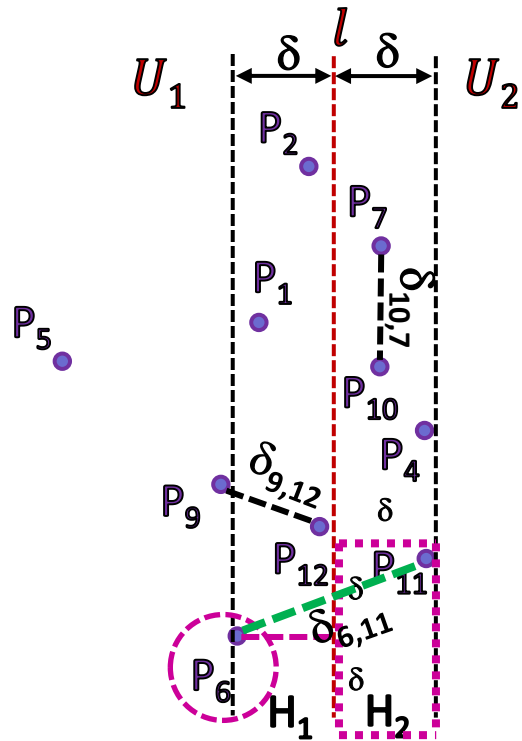


Рис.8.

4.5.1 $H_1^* = \{P_6, P_{12}, P_1, P_2\}$. Розглядаємо точку P_6 . Відкладаємо відносно неї прямокутник $\delta \times 2\delta$ у смужі H_2 , у прямокутник потрапила лише точка P_{11} із $H_2^* = \{P_{11}, P_4, P_{10}, P_7\}$. Визначаємо відстань $\delta(P_6, P_{11})$ і порівнюємо її із δ , $\delta(P_6, P_{11}) > \delta$, отже пропускаємо (рис.8).

4.5.2 $H_1^* = \{P_6, P_{12}, P_1, P_2\}$. Розглядаємо точку P_{12} (рис.9). Відкладаємо відносно неї прямокутник $\delta \times 2\delta$ у смужі H_2 . У прямокутник потрапили точки P_{11}, P_4 із $H_2^* = \{P_{11}, P_4, P_{10}, P_7\}$.

Визначаємо відстані $\delta(P_{12}, P_{11})$, $\delta(P_{12}, P_4)$ і порівнюємо їх із δ , $\delta(P_{12}, P_{11}) < \delta$, $\delta(P_{12}, P_4) > \delta$ отже запам'ятовуємо лише пару $\delta(P_{12}, P_{11})$ точок і відстань між ними.

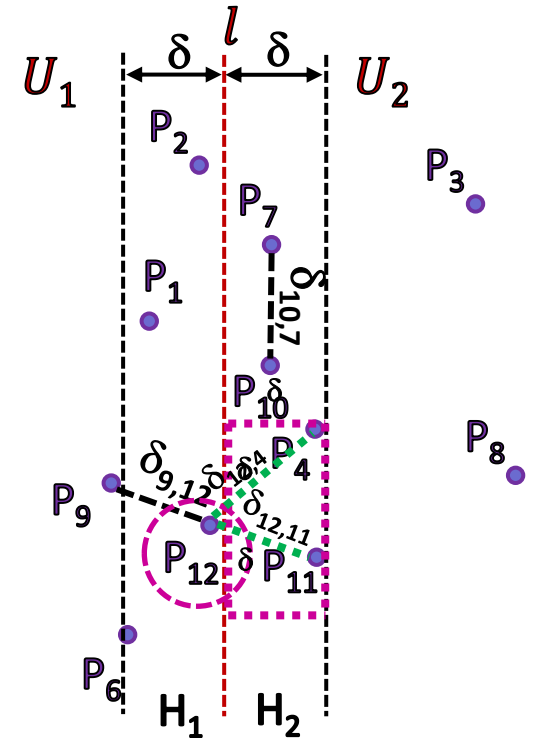


Рис.9.

Метод «розділяй та пануй» розв'язання задачі пошуку найближчої пари

4.5.3 $H_1^* = \{P_6, P_{12}, P_1, P_2\}$. Розглядаємо точку P_1 . Відкладаємо відносно неї прямокутник $\delta \times 2\delta$ у смужі H_2 . У прямокутник потрапили точки P_4, P_{10}, P_7 із $H_2^* = \{P_{11}, P_4, P_{10}, P_7\}$. Визначаємо відстані $\delta(P_1, P_4)$, $\delta(P_1, P_{10})$, $\delta(P_1, P_7)$ і порівнюємо їх із δ : $\delta(P_1, P_4) > \delta$, $\delta(P_1, P_{10}) = \delta$, $\delta(P_1, P_7) > \delta$ отже запам'ятовуємо лише пару $\delta(P_1, P_{10})$ точок і відстань між ними, (рис.10).

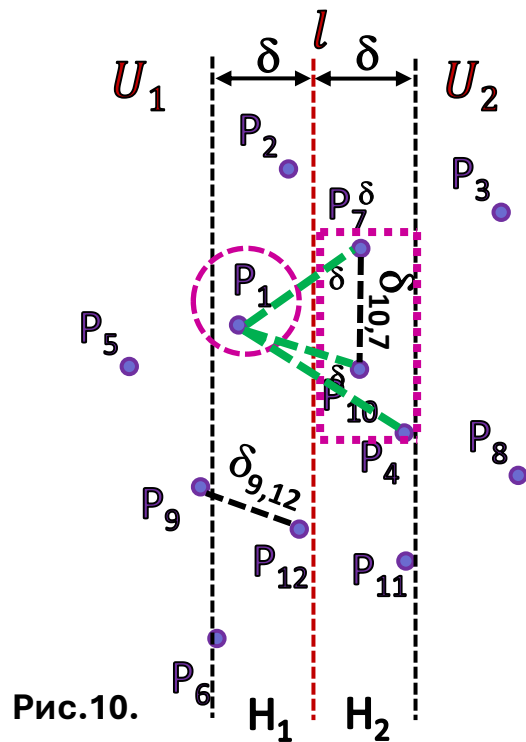


Рис.10.

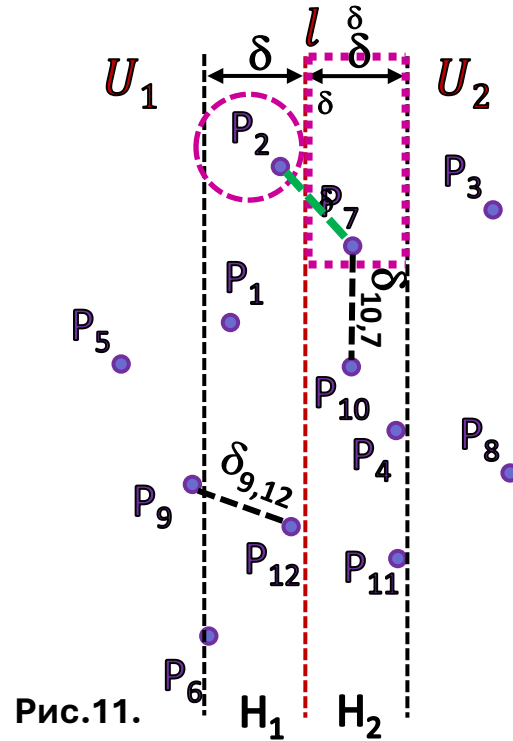


Рис.11.

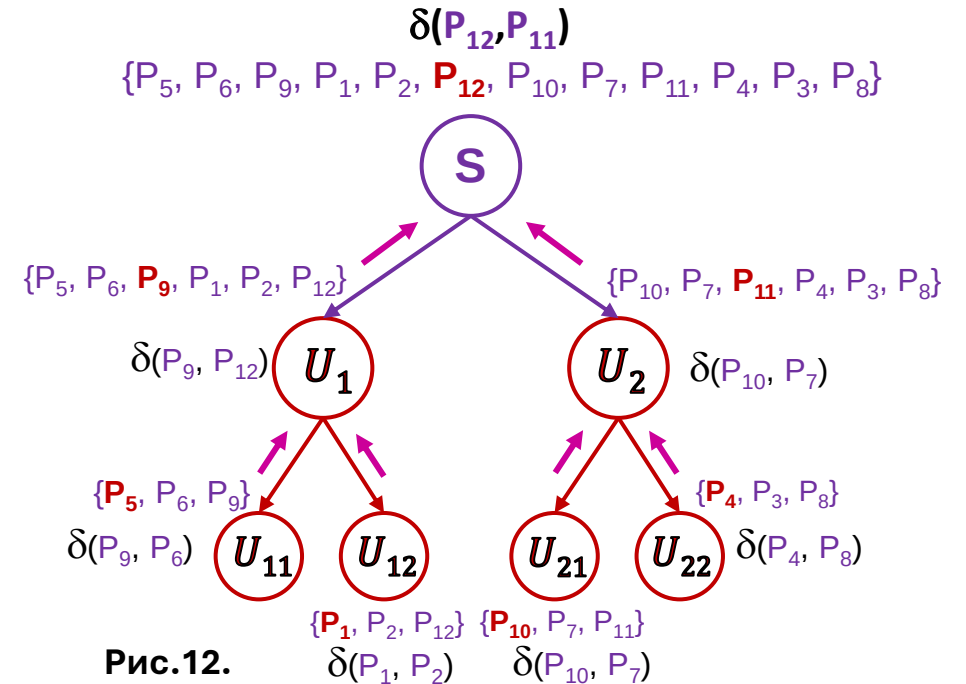


Рис.12.

4.5.4 $H_1^* = \{P_6, P_{12}, P_1, P_2\}$. Розглядаємо точку P_2 (рис.11). Відкладаємо відносно неї прямокутник $\delta \times 2\delta$ у смужі H_2 . У прямокутник потрапила лише точка P_7 із $H_2^* = \{P_{11}, P_4, P_{10}, P_7\}$. Визначаємо відстань $\delta(P_2, P_7)$ і порівнюємо її із δ : $\delta(P_2, P_7) < \delta$, отже запам'ятовуємо пару $\delta(P_2, P_7)$ точок і відстань між ними.

5. Список H_1^* пройдено. Знаходимо найближчу пару серед знайдених:

$\delta(P_{12}, P_{11}) < \delta(P_2, P_7) < \delta(P_1, P_{10}) = \delta$, отже **НП** = (P_{12}, P_{11}) з відстанню $\delta(P_{12}, P_{11})$, рис.12.

Деякі публікації за темою «задачі близькості»

1. Терещенко В.М., Наконечний Т.В., Рогожин О.Г. Застосування методів Обчислювальної геометрії до деяких задач просторового аналізу демографічної ситуації // Геоінформатика, НАНУ, Київ, № 1, 2006, С. 26-30 (ФВ).
2. Терещенко В.М., Плахтій Ю.І., Лізунова А. Г. Узагальнений алгоритм розв'язання задач обчислювальної геометрії на прикладі задачі побудови діаграми Вороного // XII-а міжнародна наукова конференція імені акад. М. Кравчука: тези доповідей. - Київ, КПІ, 2008. – С. 812-813 (тези).
3. V. Tereshchenko. One Tool for Building Visual Models // Proceedings of 2009 International Conference on Computational Intelligence, Modelling and Simulation. Brno, CZ, IEEE CS, 2009, P. 59-62, ISBN 978-0-7695-3795-5 (**Scopus**).
4. V. N. Tereshchenko, A. V. Anisimov. Recursion and parallel algorithms in geometric modeling problems. Journal: Cybernetics and Systems Analysis: Vol. 46, N 2 (2010), Page 173 - 184
5. Vasyl Tereshchenko, Anatoliy Anisimov. One Conception of creating Tools for Geometric Modeling. In proc. of 7-th International Symposium on Voronoi Diagrams in Science and Engineering (ISVD 2010), June 28-30, 2010, Quebec, Canada, IEEE-CS, pp.260-265, ISBN: 978-0- 7695-4112-9
6. Vasyl Tereshchenko, Igor Budjak, and Andrey Fisunen. The Unified Algorithmic Platform for Solving Complex Problems of Computational Geometry // In Proc. of the 12th IC “Parallel Computing Technologies” (PaCT 2013), Springer.- 2013.- P. 424-429 (**Scopus**).
7. Терещенко В.М.б Коцур Д.В.Побудова діаграми Вороного для об'єктів довільної форми // Вісник київського університету. Серія: фіз.-мат. науки.- 2014. – вип. № 1, С.145-148(**ФВ**).
8. Терещенко В.М., Танчук В.В., Терещенко Я.В. Розпізнавання та побудова контуру об'єкта на зображенні за допомогою Діаграми Вороного/ XIII міжнародна науково-практична конференція «Теоретичні та прикладні аспекти побудови програмних систем», Київ, 5-6 грудня, 2016, праці конференції, ст. 228-232.
9. В.В. Танчук, В.М. Терещенко. Розпізнавання та побудова контуру об'єкта на зображенні за допомогою Діаграми Вороного// Вісник київського університету. Серія: фіз.-мат. науки.-2017. – вип. № 1, С. 69-74(**ФВ**).
10. Kotsur,D., Tereshchenko,Y., Tereshchenko,V.: A fast approximation of the Voronoi diagram for a set of pairwise disjoint arcs //Computer Science Research Notes (Proceedings 26-th International Conference in Central Europe on Computer Graphics, Visualization and Computer Vision, Plzen, Czech Republic, May 28 – June 1, 2018 (WSCG 2018)), Plzen, Czech Republic.- 2018.-pp. 7-13.

Деякі публікації за темою «задачі близькості»

11. D. Kotsur, V. Tereshchenko, S. Vora¹, R. Windoffer, R. E. Leube, J. Mattes. VORONOI-BASED SKELETONIZATION ALGORITHM FOR SEGMENTING THE NETWORK OF INTERMEDIATE FILAMENTS/2019 IEEE International Symposium on Biomedical Imaging (ISBI). - **Hilton Molino Stucky, Venice, Italy**, April 8-11, 2019.- pp. **110-115**.
12. D. Kotsur, V. Tereshchenko . VORONOI-BASED SKELETON FOR SEGMENTING THE NETWORK OF BIOLOGICAL NEURONS/ proceedings of [The Sixth International Conference on Analytic Number Theory and Spatial Tessellations, September 24-28, 2018, Kyiv, Ukraine](#).- pp. 41-54.
13. D. Kotsur, V. Tereshchenko .An approximation of the Voronoi diagram for a set of arcs/Proceeding book of International Conference on Advanced Technologies, Computer Engineering and Science(ICATCES 2018).- 11-13 May 2018 , Safranbolu, Turkey. –pp. 620-625.
14. D. Kotsur, V. Tereshchenko . Voronoi-based skeletonization algorithm for segmenting the network of biological neurons // Radio Electronics, Computer Science, Control. - 2019.- № 1. – P.98 – 109 (We Of Science). DOI 10.15588/1607-3274-2019-1-10, e-ISSN 1607-3274, p-ISSN 2313-688X .
15. Kotsur D., Tereshchenko V. Optimization Heuristics for Computing the Voronoi Skeleton. Computational Science. ICCS 2019. Springer. Volume 11536. 2019. P.96-111. (SCOPUS) https://link.springer.com/chapter/10.1007%2F978-3-030-22734-0_8
16. Dmytro Kotsur, Vasyl Tereshchenko. AN OPTIMIZED ALGORITHM FOR COMPUTING THE VORONOI SKELETON\ / International Journal of Computing, 19(4) 2020, 542-554.
17. Cherneha, A., Zhelezniakov, D., Tytarchuk, P., & Tereshchenko, V. (2021). Segmentation of Handwritten Mathematical Matrices Using the Area Voronoi Diagram. In IEEE EUROCON 2021-19th International Conference on Smart Technologies (pp. 107-112). IEEE. Scopus.
18. Tereshchenko V.N., Marchenko A.A., Tereshchenko Y.V., Tara A.N. The optimal algorithm for dynamic support of the Voronoi Diagram for a set of points\\ Вісник київського університету. Серія: фіз.-мат. науки.-2020. – вип. № 4, С.63-69(ФВ).
19. <http://www.cs.ucf.edu/courses/cot5520/lectures.html>

ДЯКУЮ ЗА УВАГУ!